

P0 Implementation Spec

Ngày note: 2026-06-13

Tài liệu này là **bản vẽ thi công P0**. Mục tiêu là để team đọc xong biết phải tạo folder nào, script nào, API nào, schema nào, và mỗi người cần giao artifact gì. Không cần đoán.

0. Mục tiêu P0

P0 phải chứng minh được một luồng end-to-end:

```
video mẫu
-> extract keyframe/thumbnail
-> build manifest
-> build visual/text index
-> FastAPI /search
-> React UI grid
-> preview/frame neighbor
-> dry-run submit
-> log + eval nhỏ
```

P0 chưa cần:

- LangGraph/CRAG.
- Multi-agent.
- VLM rerank.
- TRAKE full.
- QA full.
- Fine-tune model.
- DRES thật.
- UI đẹp như sản phẩm thương mại.

P0 cần chạy được, rõ ràng, dễ thay stack.

1. Stack mặc định cho P0

Chốt stack mặc định để team không bị loạn:

Thành phần	P0 chọn	Lý do
Backend	FastAPI	Dễ viết API, dễ debug
Frontend	React + Vite	Dựng UI nhanh
Manifest	JSONL	Đơn giản, dễ inspect bằng text editor
Metadata local	JSONL/SQLite nhẹ	P0 chưa cần DB lớn
Visual embedding	CLIP hoặc precomputed .npy	Baseline nhanh
Vector index	FAISS local trước, Qdrant adapter sau	FAISS ít vận hành hơn; Qdrant tốt khi cần filter
Text search	BM25 local	Đủ cho title/metadata/OCR giả lập
Media serving	FastAPI static files	UI xem ảnh/video được ngay
Submit	Dry-run local log	Chưa cần DRES/BTC thật

Rule: nếu team chưa có Docker/Qdrant ổn, **đừng bắt đầu bằng Qdrant**. Bắt đầu FAISS/local trước. Nhưng code vẫn phải có `VectorIndex` interface để sau đổi sang Qdrant.

2. Cấu trúc repo đề xuất

```
ai_challenge_26_multimodel/  
  backend/  
    app/  
      main.py  
      api/  
        routes_health.py  
        routes_search.py  
        routes_media.py  
        routes_submit.py  
      core/  
        config.py  
        schemas.py  
      services/  
        search_service.py  
        media_service.py  
        submission_service.py  
      strategies/  
        base.py  
        textual_kis.py  
        visual_kis.py  
        qa.py  
        trake.py  
      retrievers/  
        base.py  
        visual_retriever.py  
        text_retriever.py  
        hybrid_retriever.py  
      indexes/  
        base.py  
        faiss_index.py  
        qdrant_index.py  
        bm25_index.py  
      embeddings/  
        base.py  
        clip_embedding.py  
        precomputed_embedding.py  
      repositories/
```

```
manifest_repository.py
log_repository.py
frontend/
  src/
    api/
      client.ts
    components/
      SearchBox.tsx
      ResultGrid.tsx
      ResultCard.tsx
      CandidateDetail.tsx
      FrameStrip.tsx
      SubmitPanel.tsx
    pages/
      SearchPage.tsx
    types/
      api.ts
  scripts/
    extract_keyframes.py
    build_manifest.py
    build_visual_index.py
    build_text_index.py
    run_eval.py
    verify_dataset.py
  data/
    raw/
      videos/
    processed/
      frames/
      thumbs/
      clips/
      manifest.jsonl
  indexes/
    visual.faiss
    visual_meta.jsonl
    bm25.pkl
  eval/
```

```
queries.jsonl
ground_truth.jsonl
runs/
logs/
submissions.jsonl
search_events.jsonl
docs/
```

Nếu chưa tạo code ngay, folder vẫn nên thống nhất trước để mỗi người đặt file đúng chỗ.

3. Contract trung tâm: `manifest.jsonl`

`manifest.jsonl` là xương sống. Mọi module phải đọc từ đây, không tự đoán path riêng.

3.0 Chuẩn mock data: V3C-compatible raw layout

Nên chọn cấu trúc kiểu V3C làm chuẩn cho mock data, vì nó rất gần bài toán video retrieval:

`video -> shot/segment -> keyframe -> thumbnail -> metadata`.

Raw mock data nên tổ chức như sau:

```
data/mock_v3c/  
  videos/  
    V001/  
      V001.mp4  
    V002/  
      V002.mp4  
  keyframes/  
    V001/  
      shot_000001_RKF.jpg  
      shot_000002_RKF.jpg  
    V002/  
      shot_000001_RKF.jpg  
  thumbnails/  
    V001/  
      shot_000001.jpg  
      shot_000002.jpg  
  msb/  
    V001.tsv  
    V002.tsv  
  info/  
    V001.json  
    V002.json
```

Ý nghĩa:

- `videos/` : video gốc.
- `keyframes/` : representative keyframe cho từng shot/segment.
- `thumbnails/` : bản nhỏ để UI load nhanh.
- `msb/` : shot boundary, chứa thời gian bắt đầu/kết thúc segment.
- `info/` : metadata như title, description, tags, source.

Nhưng không được để backend/frontend phụ thuộc trực tiếp vào cấu trúc V3C. Phải có importer:

```
V3cDataLoader -> manifest.jsonl  
AicDataLoader -> manifest.jsonl
```

Core system chỉ đọc `manifest.jsonl`. V3C chỉ là chuẩn mock/raw data để luyện và test pipeline.

Lý do chọn V3C-compatible:

- Có khái niệm shot/segment rõ, tốt cho video retrieval.
- Có keyframe + thumbnail, hợp với UI grid.
- Có metadata, hợp với BM25/text search.
- Có thể mở rộng sang TRAKE vì có timestamp/segment boundary.
- Gần format VBS, có ích để mock contest.

Nếu sau này BTC cấp data theo kiểu `Videos/`, `Keyframes/`, `Objects/`, `Metadata/`, chỉ cần viết importer mới để sinh cùng manifest schema.

Mỗi dòng là một frame/segment candidate:

```
{
  "video_id": "L01_V001",
  "frame_id": "L01_V001_000123",
  "segment_id": "L01_V001_seg_0001",
  "timestamp": 12.4,
  "frame_index": 123,
  "fps": 25.0,
  "frame_path": "data/processed/frames/L01_V001/000123.jpg",
  "thumb_path": "data/processed/thumbs/L01_V001/000123.jpg",
  "video_path": "data/raw/videos/L01_V001.mp4",
  "source": "self_cut",
  "title": "optional video title",
  "description": "optional video description",
  "ocr_text": "",
  "asr_text": "",
  "tags": []
}
```

P0 field bắt buộc:

- `video_id`
- `frame_id`

- `timestamp`
- `frame_path`
- `thumb_path`
- `video_path`
- `source`

P1/P2 mới cần đầy đủ OCR/ASR/tags.

4. Contract API

4.1 GET /health

Response:

```
{
  "status": "ok",
  "version": "p0"
}
```

4.2 POST /search

Request:

```
{
  "query": "người đàn ông đứng cạnh xe máy",
  "query_type": "auto",
  "top_k": 100,
  "filters": {
    "video_id": null
  }
}
```

`query_type` có thể là:

- `auto`
- `textual_kis`
- `visual_kis`

- qa
- trake
- ocr_heavy
- asr_heavy

P0 xử lý chắc:

- auto
- textual_kis

Các type còn lại trả được response stub hoặc fallback sang textual KIS.

Response:

```
{
  "query": "người đàn ông đứng cạnh xe máy",
  "query_type": "textual_kis",
  "strategy": "TextualKisStrategy",
  "took_ms": 82,
  "results": [
    {
      "rank": 1,
      "video_id": "L01_V001",
      "frame_id": "L01_V001_000123",
      "timestamp": 12.4,
      "score": 0.73,
      "thumb_url": "/media/thumbs/L01_V001/000123.jpg",
      "frame_url": "/media/frames/L01_V001/000123.jpg",
      "video_url": "/media/videos/L01_V001.mp4",
      "evidence": {
        "visual_score": 0.73,
        "text_score": null,
        "ocr_score": null,
        "asr_score": null,
        "cluster_score": null,
        "matched_text": []
      }
    }
  ]
}
```

4.3 GET /frames/{frame_id}/neighbors

Query params:

```
window=5
```

Response:

```
{
  "frame_id": "L01_V001_000123",
  "video_id": "L01_V001",
  "neighbors": [
    {
      "frame_id": "L01_V001_000118",
      "timestamp": 11.8,
      "thumb_url": "/media/thumbs/L01_V001/000118.jpg",
      "frame_url": "/media/frames/L01_V001/000118.jpg"
    }
  ]
}
```

4.4 GET /videos/{video_id}

Response:

```
{
  "video_id": "L01_V001",
  "video_url": "/media/videos/L01_V001.mp4",
  "title": "",
  "description": "",
  "duration": null
}
```

4.5 POST /submit/dry-run

Request:

```
{
  "query_id": "q001",
  "query": "người đàn ông đứng cạnh xe máy",
  "query_type": "textual_kis",
  "video_id": "L01_V001",
  "frame_id": "L01_V001_000123",
  "frames": [],
  "answer": null,
  "elapsed_ms": 42100,
  "user": "team_member_1"
}
```

Response:

```
{
  "status": "logged",
  "submission_id": "sub_000001"
}
```

Log vào:

```
logs/submissions.jsonl
```

5. Interface/OOP cần có

Đây là khung tư duy. Không cần over-engineer, nhưng không được viết route gọi thẳng model/index.

5.1 EmbeddingModel

```
class EmbeddingModel:
    def encode_text(self, texts: list[str]) -> "np.ndarray":
        ...

    def encode_images(self, image_paths: list[str]) -> "np.ndarray":
        ...
```

Implement P0:

- `ClipEmbeddingModel`
- hoặc `PrecomputedNpyEmbeddingModel` nếu đã có `.npy`.

5.2 VectorIndex

```
class VectorIndex:
    def build(self, vectors, metadata: list[dict]) -> None:
        ...

    def search(self, query_vector, top_k: int) -> list[dict]:
        ...

    def save(self, path: str) -> None:
        ...

    def load(self, path: str) -> None:
        ...
```

Implement P0:

- `FaissVectorIndex`

Stub:

- `QdrantVectorIndex`

5.3 TextSearchIndex

```
class TextSearchIndex:
    def build(self, docs: list[dict]) -> None:
        ...

    def search(self, query: str, top_k: int) -> list[dict]:
        ...
```

Implement P0:

- `Bm25TextSearchIndex`

5.4 Retriever

```
class Retriever:
    def retrieve(self, query_context, top_k: int) -> list[dict]:
        ...
```

Implement P0:

- VisualRetriever
- TextRetriever
- HybridRetriever

5.5 TaskStrategy

```
class TaskStrategy:
    name: str

    def retrieve(self, query_context) -> list[dict]:
        ...

    def rerank(self, candidates: list[dict], query_context) ->
list[dict]:
        ...

    def build_ui_payload(self, results: list[dict]) -> dict:
        ...

    def format_submission(self, selected_candidate: dict) -> dict:
        ...
```

Implement P0:

- TextualKisStrategy

Stub P0:

- VisualKisStrategy
- QaStrategy

- TrakeStrategy
- OcrHeavyStrategy
- AsrHeavyStrategy

P0 rule: chưa làm được strategy nào thì fallback sang TextualKisStrategy , nhưng vẫn giữ class riêng để sau này lắp vào.

5.6 SubmissionClient

```
class SubmissionClient:
    def submit(self, payload: dict) -> dict:
        ...
```

Implement P0:

- DryRunSubmitClient

Stub:

- DresSubmitClient
- BtcSubmitClient

6. Query strategy P0

6.1 TextualKisStrategy

P0 flow:

```
query
-> encode text
-> visual vector search top-100
-> optional BM25 search top-50
-> merge/fusion đơn giản
-> group/boost theo video nếu kịp
-> return UI payload
```

P0 scoring đơn giản:

```
final_score = visual_score
```

Nếu có BM25:

```
final_score = 0.8 * visual_score + 0.2 * text_score
```

Không cần perfect. Cần chạy được và dễ debug.

6.2 Stub strategy khác

QaStrategy P0:

- nhận query
- fallback search textual KIS
- response thêm field `answer_required: true`
- UI cho phép nhập answer tay

TrakeStrategy P0:

- nhận query
- fallback search textual KIS
- response thêm field `sequence_required: true`
- chưa cần DP

VisualKisStrategy P0:

- nhận query mô tả
- fallback textual KIS
- UI có thể dùng cùng grid

Mục tiêu của stub: frontend/backend không vỡ khi sau này thêm logic thật.

7. Script cần có

7.1 `scripts/extract_keyframes.py`

Input:

```
data/raw/videos/
```


Output:

```
data/processed/frames/<video_id>/*.jpg  
data/processed/thumbs/<video_id>/*.jpg
```

Yêu cầu:

- Mặc định extract 1 fps.
- Có option `--fps`.
- Tên frame ổn định: `000001.jpg`, `000002.jpg`.

7.2 `scripts/build_manifest.py`

Input:

- `data/raw/videos/`
- `data/processed/frames/`
- `data/processed/thumbs/`

Output:

```
data/processed/manifest.jsonl
```

Yêu cầu:

- Mỗi frame một dòng JSON.
- Validate path tồn tại.
- In summary: số video, số frame, số lỗi.

7.3 `scripts/build_visual_index.py`

Input:

```
data/processed/manifest.jsonl
```

Output:

```
indexes/visual.faiss
indexes/visual_meta.jsonl
```

Yêu cầu:

- Encode frame hoặc load precomputed vector.
- Metadata trong `visual_meta.jsonl` phải map cùng thứ tự với vector.

7.4 `scripts/build_text_index.py`

Input:

```
data/processed/manifest.jsonl
```

Output:

```
indexes/bm25.pkl
```

P0 text fields:

- title
- description
- ocr_text nếu có
- asr_text nếu có

7.5 `scripts/run_eval.py`

Input:

```
eval/queries.jsonl
eval/ground_truth.jsonl
```

Output:

```
eval/runs/<timestamp>.json
```

Metric P0:

- Recall@10
- Recall@50
- Recall@100
- avg latency
- p95 latency

8. Eval schema

8.1 eval/queries.jsonl

```
{  
  "query_id": "q001",  
  "query_type": "textual_kis",  
  "query": "người đàn ông đứng cạnh xe máy",  
  "notes": "query giả lập"  
}
```

8.2 eval/ground_truth.jsonl

```
{  
  "query_id": "q001",  
  "video_id": "L01_V001",  
  "frame_id": "L01_V001_000123",  
  "acceptable_frames": [  
    "L01_V001_000122",  
    "L01_V001_000123",  
    "L01_V001_000124"  
  ],  
  "answer": null  
}
```

Với P0, ground truth có thể làm thủ công 30-50 câu là đủ.

9. Frontend P0 chi tiết

9.1 Màn hình chính

Layout tối thiểu:

```
-----  
Search box           [Search] [query type]  
-----  
Left/main: result grid  
Right: candidate detail  
Bottom/right: submit panel  
-----
```

9.2 Component cần có

SearchBox

- input query
- select query type: auto/textual_kis/qa/trake
- button search

ResultGrid

- nhận `results`
- render thumbnail cards
- click card -> chọn candidate

ResultCard

- thumbnail
- rank
- score
- video_id
- timestamp

CandidateDetail

- frame lớn
- video preview nếu có
- evidence scores
- metadata text

FrameStrip

- gọi `/frames/{frame_id}/neighbors`
- hiện frame trước/sau
- click neighbor để chọn candidate

SubmitPanel

- hiện output format
- nút dry-run submit
- nếu query type là QA thì có ô answer

9.3 Rule UI P0

- Không cần đẹp, cần nhanh.
- Thumbnail phải load được.
- Candidate đang chọn phải rõ.
- Submit không quá 2 click sau khi thấy candidate.
- Nếu API lỗi, hiện lỗi rõ.

10. Chia task theo thứ tự build

Ngày 1: Contract và skeleton

Done cuối ngày:

- Repo folder structure có đủ.
- `manifest.jsonl` schema chốt.
- API schema chốt.
- Frontend dùng mock data hiển thị grid được.

Người làm:

- Backend: tạo FastAPI skeleton.
- Frontend: tạo Vite app + mock grid.
- Data: chuẩn bị 5-10 video mẫu.
- Retrieval: chuẩn bị script build index skeleton.
- Eval: tạo 10 query giả lập đầu tiên.

Ngày 2: Data thật vào UI

Done cuối ngày:

- Extract được keyframe/thumb.
- Build được manifest.
- Backend serve media.
- UI load thumbnail từ backend.

Ngày 3: Search thật

Done cuối ngày:

- Build visual index.
- `/search` trả top candidates thật.
- UI gọi `/search` và render kết quả thật.

Ngày 4: Candidate detail và submit

Done cuối ngày:

- Click candidate xem frame/video.
- Có frame neighbor.
- Có `/submit/dry-run`.
- Submit log vào `logs/submissions.jsonl`.

Ngày 5: Eval và mock contest nhỏ

Done cuối ngày:

- Có 30 query giả lập.
- `run_eval.py` đo Recall@K.
- Mock contest 10 câu, ghi lỗi UI/search.

Ngày 6-7: Fix và đóng P0

Done cuối ngày 7:

- P0 demo ổn định.
- Có báo cáo metric.
- Có backlog P1 dựa trên lỗi thật.

11. Definition of done P0

P0 chỉ được gọi là xong khi đủ các dòng này:

- ☐ Có ít nhất 20 video mẫu hoặc dataset nhỏ tương đương.
- ☐ Chạy được `extract_keyframes.py`.
- ☐ Chạy được `build_manifest.py`.
- ☐ Có `data/processed/manifest.jsonl`.
- ☐ Chạy được `build_visual_index.py`.
- ☐ Backend `GET /health` ok.
- ☐ Backend `POST /search` trả top-100.
- ☐ Backend serve được thumbnail/frame/video.
- ☐ UI search được.
- ☐ UI hiện grid thumbnail.
- ☐ UI click candidate xem detail được.
- ☐ UI xem frame neighbor được.
- ☐ UI dry-run submit được.
- ☐ Log submit được ghi.
- ☐ Có 30-50 query eval.
- ☐ `run_eval.py` in Recall@10/50/100 và latency.

12. Những lỗi thường gặp và cách tránh

Lỗi 1: Mỗi người tự đặt schema riêng

Cách tránh:

- Chỉ dùng `manifest.jsonl` làm source of truth.
- Không tự invent field mới trong frontend/backend mà không cập nhật spec.

Lỗi 2: Route `/search` làm quá nhiều thứ

Cách tránh:

- Route chỉ parse request và gọi `SearchService`.
- `SearchService` gọi `TaskStrategy`.
- `TaskStrategy` gọi retriever/reranker.

Lỗi 3: Dính cứng vào FAISS/CLIP

Cách tránh:

- Business logic nhận `VectorIndex` , không gọi trực tiếp `faiss.Index` .
- Business logic nhận `EmbeddingModel` , không gọi trực tiếp CLIP.

Lỗi 4: UI chờ backend hoàn chỉnh mới làm

Cách tránh:

- Frontend làm mock data từ ngày 1.
- Khi backend có `/search` , chỉ đổi client API.

Lỗi 5: Eval để sau

Cách tránh:

- Ngày 1 đã tạo 10 query.
- Ngày 5 phải có 30 query và run eval.

13. Quy tắc ưu tiên

Nếu thiếu thời gian, làm theo thứ tự:

1. Manifest.
2. Media serving.
3. Visual search.
4. `/search` .
5. UI grid.
6. Candidate detail.
7. Dry-run submit.
8. Eval.
9. Text/BM25.
10. Group-by-video.

Không đảo thứ tự để đi làm agent/reranker sớm.

14. P0 output cuối cùng đem họp/demo

Cuối P0 phải demo được:

1. Chạy backend.

2. Mở frontend.
3. Gõ query.
4. Thấy grid kết quả.
5. Click một candidate.
6. Xem frame/video và frame lân cận.
7. Dry-run submit.
8. Mở log thấy submission.
9. Chạy eval thấy Recall@K/latency.

Nếu demo được 9 bước này, P0 đạt. Sau đó mới lắp OCR, ASR, TRAKE, QA, reranker, DRES/BTC client.